

Pixels and Filters (continue)

DO Thanh Ha

Informatics Departement
Mathematics - Mechanics - Informatics
VNU University of Science

- Images as functions
- Point Operators
- Linear System (filters)
- Convolution and Correlation

Point Operations

- Point operations changes a pixel's intensity value according to some function (don't care about pixel's neighbor)
- Also called a **homogeneous operation**
- New pixel intensity **depends on**
 - Pixel's previous intensity $I(u, v)$
 - Mapping function $f()$
- **Does not depend on**
 - Pixel's location (u, v)
 - Intensities of neighboring pixels

Some Homogeneous Point Operations

- Addition (changes brightness): $f(p) = p + k$
 - E.g. $f_{\text{bright}}(p) = p + 10$
- Multiplication (stretches/shrinks image contrast range):
 $f(p) = p \times k$
 - E.g. $f_{\text{contrast}}(p) = p \times 1.5$
- Real-valued functions: $\exp(x)$, $\log(x)$, $(1/x)$, x^k , ...
- Quantizing pixel values
- Global thresholding
- Gamma correction

Non-Homogeneous Point Operation

- New pixel value depends on:
 - Old value + pixel's location (u,v)

$$a' \leftarrow g(a, u, v) \quad (1)$$

$$I'(u, v) \leftarrow g(I(u, v), u, v) \quad (2)$$

- Deals with pixel values outside displayable range
 - if($a > 255$) $a = 255$;
 - If($a < 0$) $a = 0$;
- Function below will clamp (force) all values to fall within range $[a,b]$

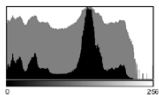
$$f(p) = \begin{cases} a, & \text{if } p < a \\ p, & \text{if } a \leq p \leq b \\ b, & \text{if } p > b \end{cases}$$

Inverting Images

$$f_{invert}(a) = -a + a_{max} = a_{max} - a$$

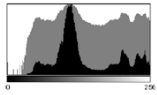
- 2 steps

- Multiple intensity by -1
- Add constant (e.g. a_{max}) to put result in range $[0, a_{max}]$



(a)

Original

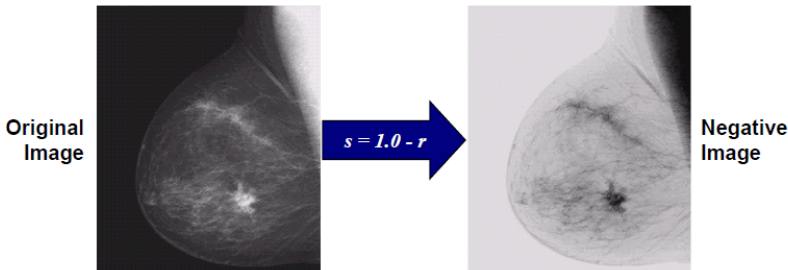


(c)

Inverted Image

Image Negatives (Inverted Images)

- Image negatives useful for enhancing white or grey detail embedded in dark regions of an image
 - Note how much clearer the tissue is in the negative image of the mammogram below



Thresholding

- Input values below **threshold** a_{th} set to a_0 and input values above **threshold** a_{th} set to a_1

$$f_{threshold}(a) = \begin{cases} a_0, & \text{for } a < a_{th} \\ a_1, & \text{for } a \geq a_{th} \end{cases}$$

- Converts grayscale image to binary image (binarization) if $a_0 = 0, a_1 = 1$



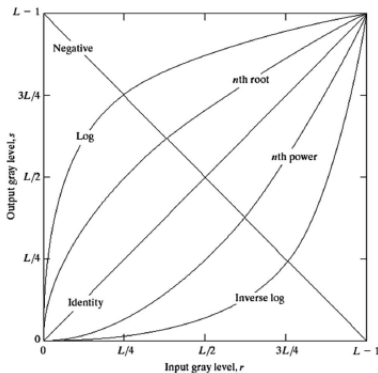
Original Image



Thresholded Image

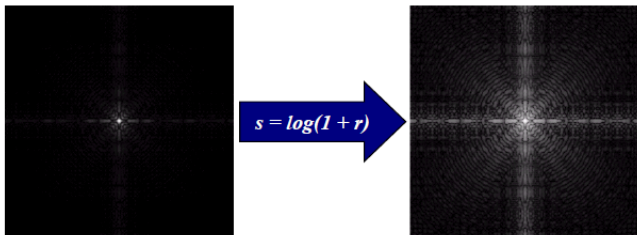
Basic Grey Level Transformations

- 3 most common gray level transformation:
 - Linear: Negative/Identity
 - Logarithmic: Log/Inverse log
 - Power law: n^{th} power/ n^{th} root



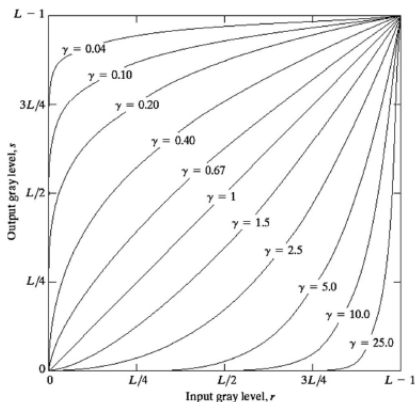
Logarithmic Transformations

- Maps narrow range of input levels $\Rightarrow$ wider range of output values
- Inverse log transformation does opposite transformation
- The general form of the log transformation is
$$s = c \times \log(1 + r)$$
- Log transformation of Fourier transform shows more detail



Power Law Transformations

- Power law transformations have the form $s = c \times r^\gamma$
- Map narrow range of dark input values into wider range of output values or vice versa
- Varying γ gives a whole family of curves



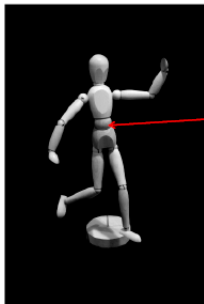
Intensity Windowing

- A clamp operation, then linearly stretching image intensities to fill possible range
- To window an image in $[a, b]$ with max intensity **M**

$$f(p) = \begin{cases} 0, & \text{if } p < a \\ M \times \frac{p-a}{b-a}, & \text{if } a \leq p \leq b \\ M, & \text{if } p > b \end{cases}$$



Original Image

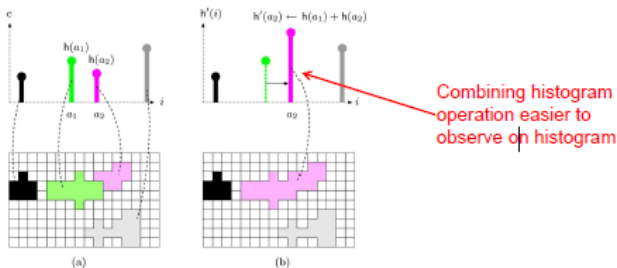


Windowed Image

Contrasts
easier to see

Point Operations and Histograms

- Effect of some point operations easier to observe on histograms
 - Increasing brightness
 - Raising contrast
 - Inverting image
- Operations that merge histogram bins are irreversible



Automatic Contrast Adjustment

- Point operation that modifies pixel intensities such that available range of value is fully converted
- Algorithm:
 - Find high and lowest pixel intensities a_{low} , a_{high}
 - Linear stretching of intensity range



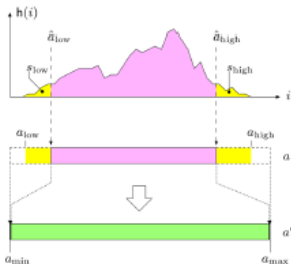
$$f_{ac}(a) = a_{min} + (a - a_{low}) \cdot \frac{a_{max} - a_{min}}{a_{high} - a_{low}}$$

If $a_{min} = 0$ and $a_{max} = 255$

$$f_{ac}(a) = (a - a_{low}) \cdot \frac{255}{a_{high} - a_{low}}$$

Modified Contrast Adjustment

- Better to map only certain range of values
- Get rid of tails (usually noise) based on predefined percentiles (s_{low} , s_{high})



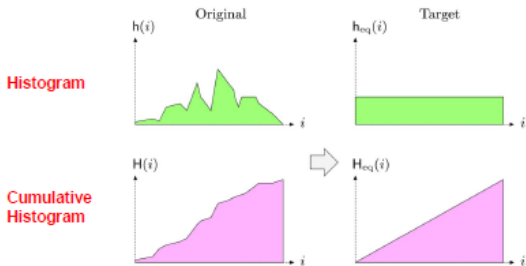
$$\hat{a}_{low} = \min\{i \mid H(i) \geq M \cdot N \cdot s_{low}\}$$

$$\hat{a}_{high} = \max\{i \mid H(i) \leq M \cdot N \cdot (1 - s_{high})\}$$

$$f_{mac}(a) = \begin{cases} a_{min} & \text{for } a \leq \hat{a}_{low} \\ a_{min} + (a - \hat{a}_{low}) \cdot \frac{a_{max} - a_{min}}{\hat{a}_{high} - \hat{a}_{low}} & \text{for } \hat{a}_{low} < a < \hat{a}_{high} \\ a_{max} & \text{for } a \geq \hat{a}_{high} \end{cases}$$

Histogram Equalization

- Adjust 2 different images to make their histograms (intensity distributions) similar
- Apply a point operation that changes histogram of modified image into uniform distribution
-



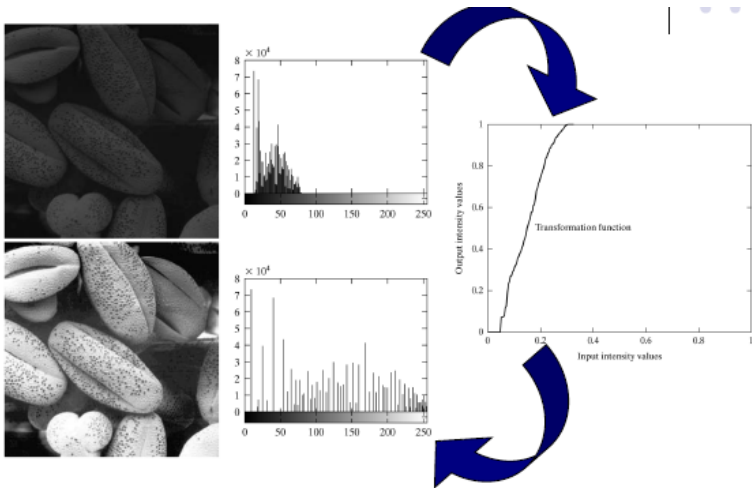
Histogram Equalization

- Spreading out the frequencies in an image (or equalizing the image) is a simple way to improve dark or washed out images
- Can be expressed as a transformation of histogram

$$s_k = T(r_k)$$

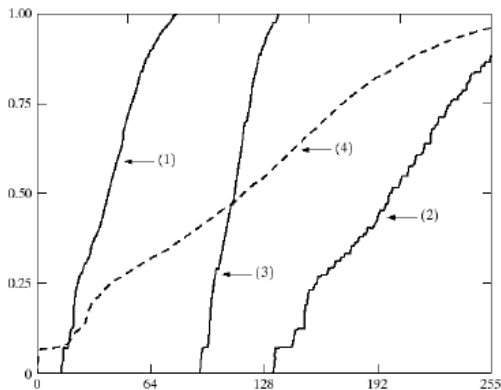
- r_k : input intensity
- s_k : processed intensity
- k : the intensity range (e.g. 0.0 - 1.0)

Equalization Transformation Function

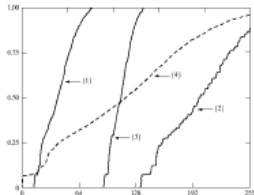
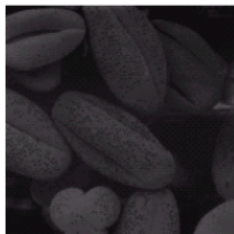
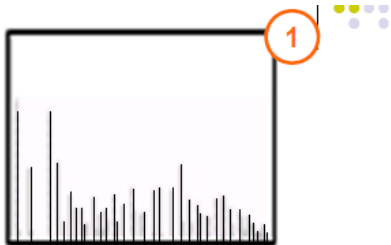
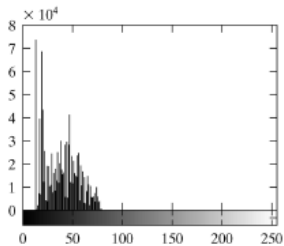


Equalization Transformation Function

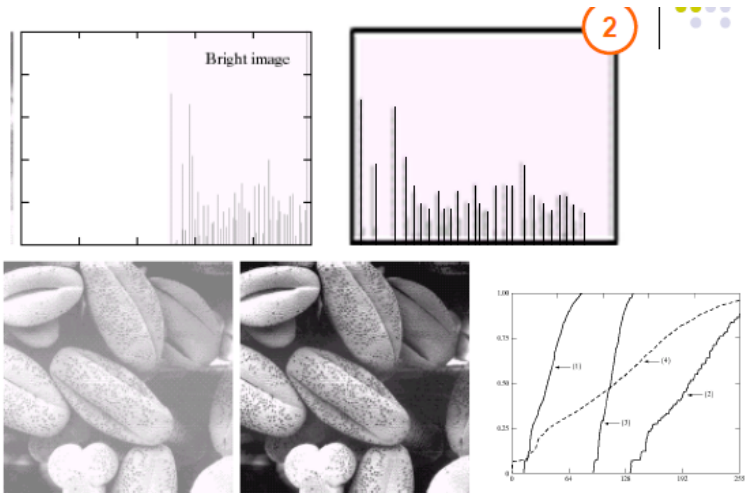
- Different equalization function (1-4) may be used



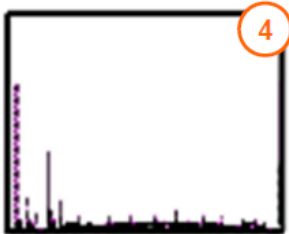
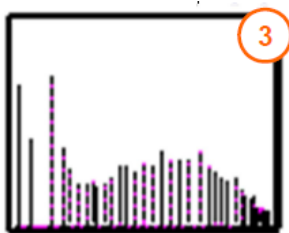
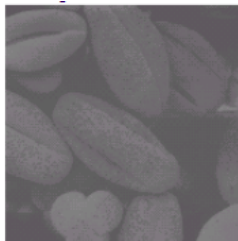
Equalization Examples



Equalization Examples



Equalization Examples



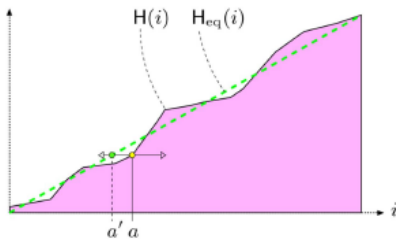
Linear Histogram Equalization

- Histogram cannot be made exactly flat – peaks cannot be increased or decreased by point operations.
- Following point operation makes histogram as flat as possible: (assuming $M \times N$ image and pixels in range $[0, K - 1]$)

Point operation that returns
Linear equalized value of a

$$f_{eq}(a) = \left\lfloor H(a) \cdot \frac{K-1}{MN} \right\rfloor$$

Cumulative Histogram: Σ how many
times intensity a occurs



Effects of Linear Histogram Equalization

Original Image I



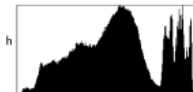
(a)

Image I' after Linear Equalization



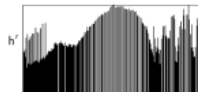
(b)

Original histogram



(c)

Histogram after Linear Equalization



(d)

Cumulative Histogram



(e)

Cumulative Histogram After Linear Equalization



(f)

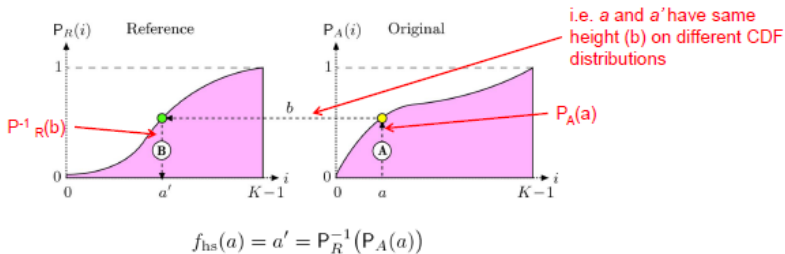
Histogram Specification

- Real images never show uniform distribution (unnatural)
- Most real images, distribution of pixel intensities is gaussian
- Histogram specification
 - modifies an image's histogram into an arbitrary intensity distribution (may not be uniform)
- Image 1's histogram can also be used as target for image 2
 - Why? Makes images taken by 2 different cameras to appear as if taken by same camera

- Histograms can be interpreted as probabilities
 - Question: If I pick a pixel from an image at random, what is the probability that the pixel has intensity i ?
 - Answer: $P(I(u, v) = i) = \frac{\# \text{ pixels with value } i}{\text{total } \# \text{ pixel}}$
or in terms of the histogram h
 $P(I(u, v) = i) = h(i)/wh$

Histogram Specification

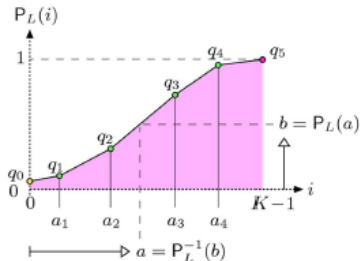
- Find a mapping such that distribution of a matches some reference distribution. i.e $a' = f_{hs}(a)$ to convert original image I_A into $I_{A'}$ such that



Adjusting Linear Distribution Piecewise

- In practice, reference distribution may be specified as a piecewise linear function

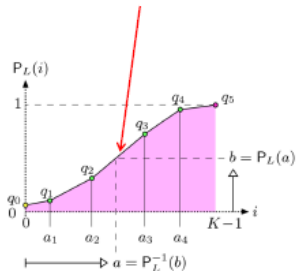
$$\mathcal{L} = [\langle a_0, q_0 \rangle, \langle a_1, q_1 \rangle, \dots, \langle a_k, q_k \rangle, \dots, \langle a_N, q_N \rangle]$$



- 2 endpoints are fixed $\langle 0, q_0 \rangle$ and $\langle K-1, 1 \rangle$

Adjusting Linear Distribution Piecewise

For each segment, linearly interpolate to find any value



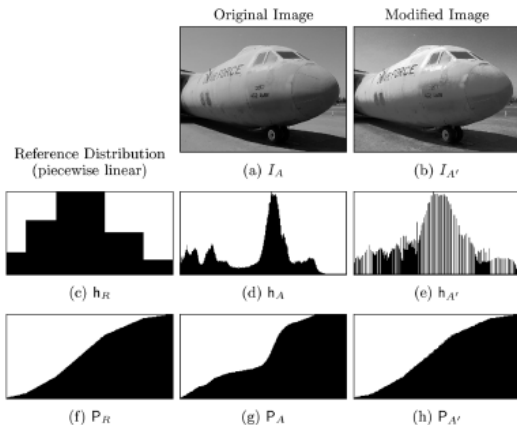
$$P_L(i) = \begin{cases} q_m + (i - a_m) \cdot \frac{(q_{m+1} - q_m)}{(a_{m+1} - a_m)} & \text{for } 0 \leq i < K-1 \\ 1 & \text{for } i = K-1 \end{cases}$$

$$P_L^{-1}(b) = \begin{cases} 0 & \text{for } 0 \leq b < P_L(0) \\ a_n + (b - q_n) \cdot \frac{(a_{n+1} - a_n)}{(q_{n+1} - q_n)} & \text{for } P_L(0) \leq b < 1 \\ K-1 & \text{for } b \geq 1 \end{cases}$$

We also need the inverse mapping

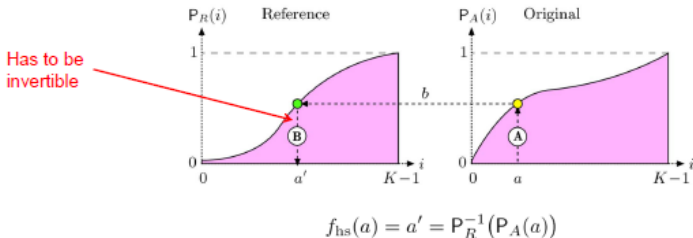
$$n = \max\{j \in \{0, \dots, N-1\} \mid q_j \leq b\}$$

Adjusting Linear Histogram Piecewise



Histogram Matching

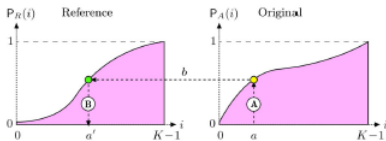
- Prior method needed reference distribution to be invertible



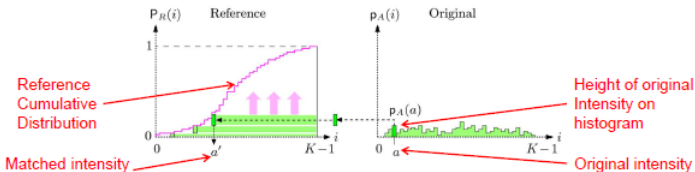
- What if reference histogram is not invertible?
- For example not invertible if histogram has some intensities that occur with probability 0? i.e. $p(k) = 0$
- Use different method called **histogram matching**

Histogram Matching

- Given two images I_A and I_B , we want to make their intensity profiles look as similar as possible
- How: Match their cumulative histogram H_A and H_B



$$f_{hs}(a) = a' = \min\{j \mid (0 \leq j < K) \wedge (P_A(a) \leq P_R(j))\}$$



Adjusting to a Given Histogram

Target Image



(a) I_A

Reference Image

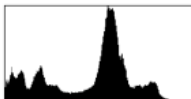


(b) I_R

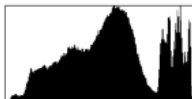
Modified Image



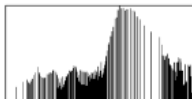
(c) $I_{A'}$



(d) h_A



(e) h_R



(f) $h_{A'}$



(g) H_A



(h) H_R



(i) $H_{A'}$

- Self Study

Filters

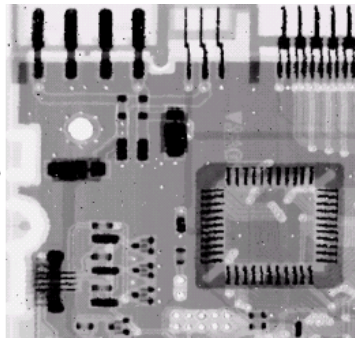
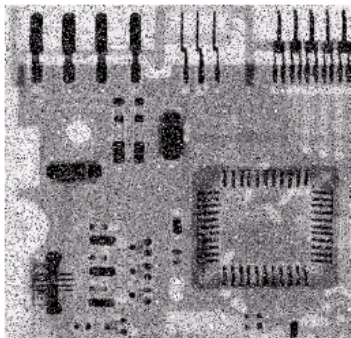
What Is Image Enhancement?

- Image enhancement makes images more useful by:
 - Highlighting interesting detail in images
 - Removing noise from images
 - Making images more visually appealing

Image Enhancement Examples



Image Enhancement Examples



Spatial & Frequency Domains

- There are two broad categories of image enhancement techniques
 - Spatial domain techniques
 - Direct manipulation of image pixels (intensity values)
 - Frequency domain techniques
 - Manipulation of Fourier transform or wavelet transform of an image
- First spatial domain techniques
- Later: frequency domain techniques

What is Filters?

- Capabilities of point operations are limited
- **Filters:** combine **pixel's value** + **values of neighbors**
- **E.g blurring:** Compute average intensity of block of pixels



- Combining multiple pixels needed for certain operations:
 - Blurring, Smoothing
 - Sharpening

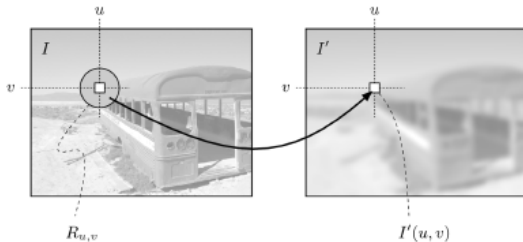
What Point Operations Can't Do

- Example: sharpening

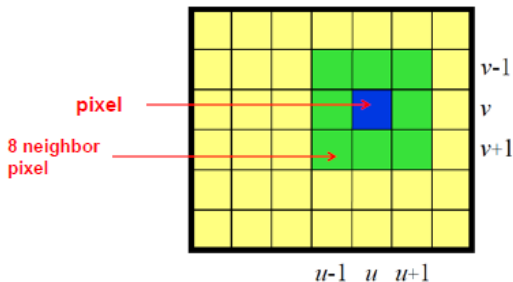


Definition: Spatial Filter

- An image operation that combines each pixel's intensity $I(u, v)$ with that of neighboring pixels
- E.g: averaged/weighted average of group pixels



Example: Average (Mean) of 3x3 Neighborhood

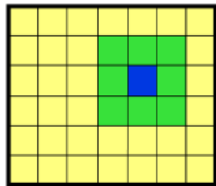
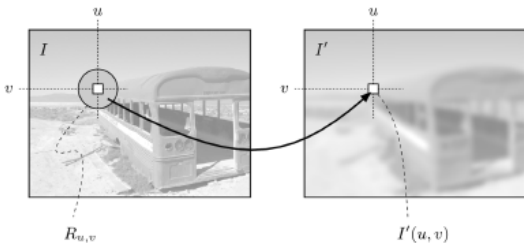


Blurring: Replace each pixel with AVERAGE Intensity of pixel + neighbors

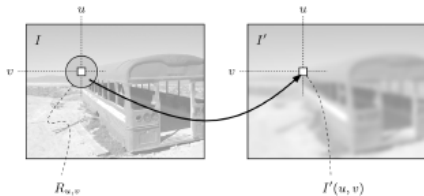
Smoothing an Image by Averaging

- Replace each pixel by average of pixel + neighbors
- For 3x3 neighborhood

$$I'(u, v) = \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$



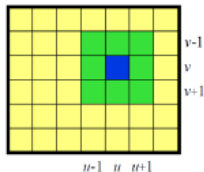
Smoothing an Image by Averaging



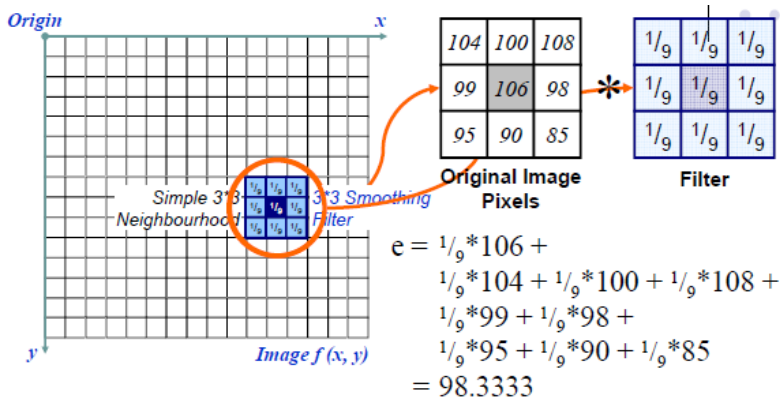
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

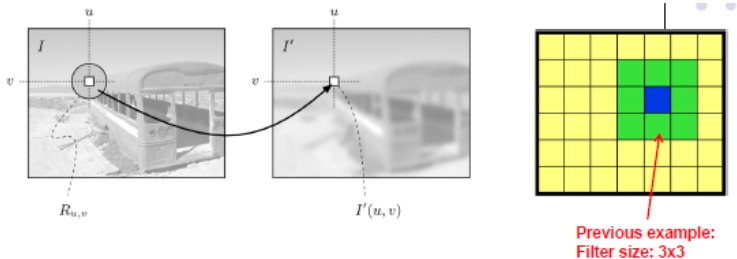


Example: Smoothing Spatial Filtering



The above is repeated for every pixel in the original image to generate the smoothed image

Smoothing an Image by Averaging



- Many possible filter parameters (size, weights, function, etc)
- **Filter size (size of neighborhood):** 3×3 , 5×5 , 7×7 , $\dots$, 21×21 , $\dots$
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels **Filters function:** can be linear (a weighted summation) or nonlinear

The Filter Matrix

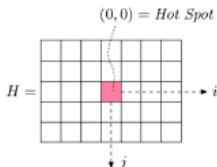
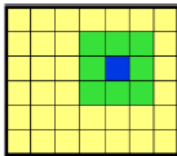
$$I'(u, v) = \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$H(i, j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filter operation can be expressed as a matrix
Example: averaging filter

Filter matrix also called
filter mask $H(i, j)$



Example: What does this Filter Do?



0	0	0
0	1	0
0	0	0



Identify function (leaves image alone)

What does this Filter Do


$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



Mean (averages neighborhood)

Mean Filters: Effect of Filter Size



Original



7×7



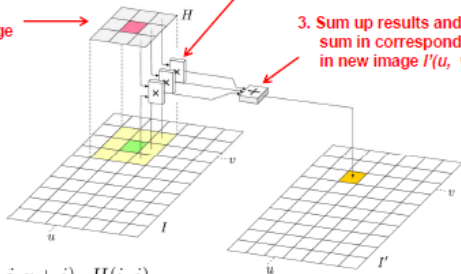
15×15



41×41

Applying Linear Filters: Convolution

- For each image position $l(u,v)$:
1. Move filter matrix H over image such that $H(0,0)$ coincides with current image position (u,v)
 2. Multiply all filter coefficients $H(i,j)$ with corresponding pixel $I(u+i, v+j)$
 3. Sum up results and store sum in corresponding position in new image $I'(u, v)$



Stated formally:

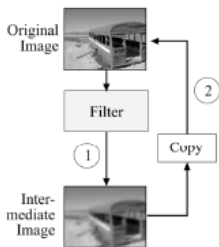
$$I'(u, v) \leftarrow \sum_{(i,j) \in R_H} I(u+i, v+j) \cdot H(i, j)$$

R_H is set of all pixels covered by filter.
For 3x3 filter, this is:

$$I'(u, v) \leftarrow \sum_{i=-1}^{i=1} \sum_{j=-1}^{j=1} I(u+i, v+j) \cdot H(i, j)$$

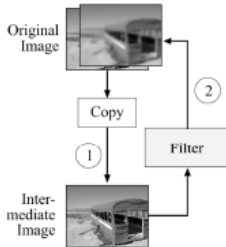
Computing Filter Operation

- Filter matrix H moves over each pixel in original image I to compute corresponding pixel in new image I'
- Cannot overwrite new pixel value in original image I Why?



Version A

Store results I' in intermediate image, then copy back to replace I



Version B

Copy original image I to intermediate image, use it as source, then store results I' to replace original image

Weighted Smoothing Filters

- More effective smoothing filters can be generated by allowing different pixels in the neighbourhood different weights in the averaging function
 - Pixels closer to central pixel more important
 - Often referred to as a weighted averaging

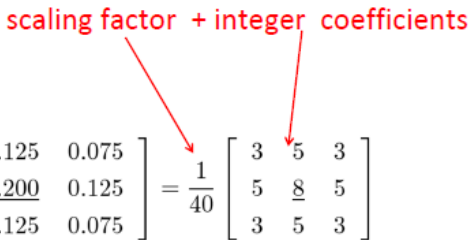
$1/16$	$2/16$	$1/16$
$2/16$	$4/16$	$2/16$
$1/16$	$2/16$	$1/16$

Weighted
averaging filter

Integer Coefficients

- Instead of floating point coefficients, more efficient, simpler to use:

scaling factor + integer coefficients

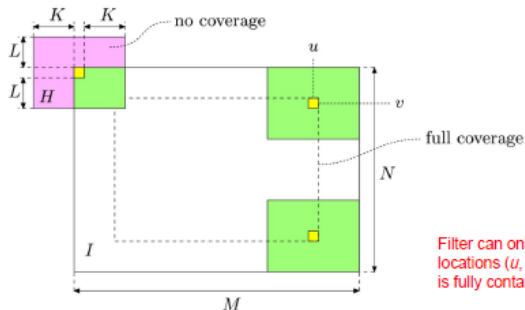
$$H(i, j) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & \underline{0.200} & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix} = \frac{1}{40} \begin{bmatrix} 3 & 5 & 3 \\ 5 & \underline{8} & 5 \\ 3 & 5 & 3 \end{bmatrix}$$


$$H(i, j) = s \cdot H'(i, j)$$

Computation Range

- For a filter of size $(2K + 1) \times (2L + 1)$, if image size is $M \times N$, filter is computed over the range:

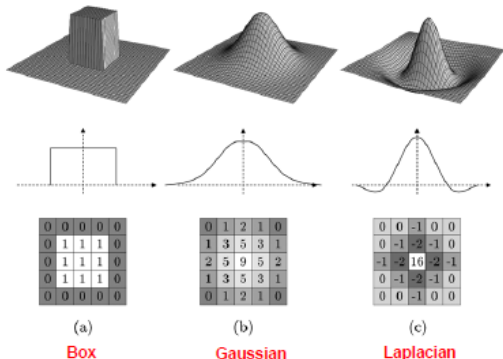
$$K \leq u' \leq (M-K-1) \quad \text{and} \quad L \leq v' \leq (N-L-1)$$



Filter can only be applied at image locations (u, v) where filter matrix H is fully contained in the image

Linear Filters: Smoothing Filters

- 2 main classes of linear filters:
 - Smoothing: +ve coefficients (weighted average). E.g box, gaussian
 - Difference filters: +ve and -ve weights. E.g. Laplacian

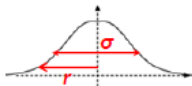
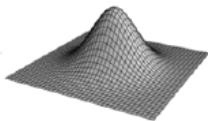


Gaussian Filter

$$G_{\sigma}(r) = e^{-\frac{r^2}{2\sigma^2}} \quad \text{or} \quad G_{\sigma}(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- where

- σ is width (standard deviation)
- r is distance from center



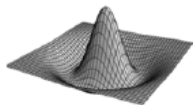
0	1	2	1	0
1	3	5	3	1
2	5	9	5	2
1	3	5	3	1
0	1	2	1	0

Gaussian
filter

Difference Filter

- **Coefficients:** some +ve, some negative
- Example: Laplacian filter
- Computation is difference

$$\sum (+ve \text{ coefficients}) - \sum (-ve \text{ coefficients})$$



$$I'(u, v) = \sum_{(i,j) \in R_H^+} I(u+i, v+j) \cdot |H(i, j)| \\ - \sum_{(i,j) \in R_H^-} I(u+i, v+j) \cdot |H(i, j)|$$

0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

Laplacian
filter

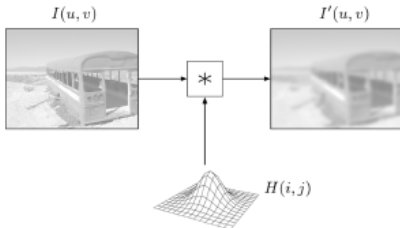
Mathematical Properties of Convolution

- Applying a filter as described called linear convolution
- For discrete 2D signal, convolution defined as:

$$I'(u, v) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} I(u-i, v-j) \cdot H(i, j)$$

Formal definition:
Sum to $\pm \infty$

$$I' = I * H$$



Properties of Convolution

- Commutativity

$$I * H = H * I$$

- Linearity

$$(s \cdot I) * H = I * (s \cdot H) = s \cdot (I * H)$$

$$(I_1 + I_2) * H = (I_1 * H) + (I_2 * H)$$

$$(b + I) * H \neq b + (I * H)$$

- Associativity

$$A * (B * C) = (A * B) * C$$

- Separability

$$H = H = H_1 * H_2 * \dots * H_n$$

$$I * H = I * (H_1 * H_2 * \dots * H_n) = (\dots((I * H_1) * H_2 * \dots * H_n))$$

- If a kernel H can be separated into multiple smaller kernels

$$H = H_1 * H_2 * \dots * H_n$$

Separability in x and y

- Sometimes we can separate a kernel into “vertical” and “horizontal” components
- Consider the kernels

$$H_x = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix}, H_y = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

then

$$H = H_x * H_y = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Gaussian Kernel

- 1D

$$g_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

- 2D

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x^2 + y^2)}{2\sigma^2}\right)$$

- 2D gaussian is just product of 1D gaussians:

$$G_{\sigma}(x, y) = g_{\sigma}(x) \cdot g_{\sigma}(y)$$

- Consequently, convolution with a gaussian is separable

$$I * G = I * G_x * G_y$$

- Where G is the 2D discrete gaussian kernel;
- G_x is “horizontal” and G_y is “vertical” 1D discrete Gaussian kernels

- Digital Image Processing (CS/ECE 545), Computer Science Dept. Worcester Polytechnic Institute (WPI)